

SonarQube Interview Questions and Answers (Java/Spring Boot)

1. What is SonarQube?

Answer: SonarQube is a static code analysis platform that analyzes source code for bugs, code smells, vulnerabilities, duplicated code, and code coverage (using external coverage tools like JaCoCo). It also enforces quality gates before deployment.

2. What is Static Code Analysis?

Answer:

Static code analysis means analyzing source code **without executing the application**.

Example:

```
String s = null;  
System.out.println(s.length());
```

The compiler allows this code because it's syntactically correct, but SonarQube reports a possible `NullPointerException`.

3. Difference between Compiler and SonarQube?

Compiler	SonarQube
Checks syntax	Checks code quality
Produces bytecode	Produces quality report
Stops on compile errors	Reports maintainability and security issues
Doesn't check coding standards	Enforces coding rules

4. What is a Code Smell?

A code smell is a maintainability issue.

Example:

```
if(flag == true)
```

Better:

```
if(flag)
```

The code works, but it is not clean or maintainable.

5. Difference between Bug and Code Smell?

Bug

A bug may break the application.

Example:

```
list.add();
```

when `list` is `null`.

Code Smell

The application works correctly, but the code quality is poor.

6. What is a Vulnerability?

A confirmed security issue.

Examples:

- SQL Injection
 - Hardcoded passwords
 - Weak encryption
-

7. What is a Security Hotspot?

A piece of code that **might** have security implications and requires manual review.

Example:

```
new Random()
```

Using `Random` for generating security tokens should typically be replaced with `SecureRandom`.

8. What is Technical Debt?

Technical debt is the estimated effort required to fix all identified issues.

Example:

```
Technical Debt  
  
6 hours
```

Meaning approximately six hours are needed to fix the reported issues.

9. What is a Quality Gate?

A Quality Gate is a set of conditions that determines whether code is acceptable.

Example:

```
Coverage > 80%  
Critical Bugs = 0  
Blocker Issues = 0
```

If any condition fails, the pipeline stops.

10. What is a Quality Profile?

A Quality Profile is a collection of coding rules used during analysis.

Example:

```
Java Profile
650 Rules Enabled
```

Organizations can customize these rules.

11. Difference between Quality Gate and Quality Profile?

Quality Profile	Quality Gate
Defines analysis rules	Defines pass/fail criteria
Used during analysis	Evaluated after analysis
Answers "What should be checked?"	Answers "Can the build continue?"

12. Why do we need JaCoCo?

SonarQube does not generate code coverage.

JaCoCo executes unit tests and produces:

```
jacoco.xml
```

SonarQube imports this report to display coverage.

13. Does SonarQube execute Unit Tests?

No.

- JUnit executes the tests.
- JaCoCo measures coverage.
- SonarQube imports the generated coverage report.

14. Can SonarQube fail a Jenkins build?

Yes.

Example:

```
waitForQualityGate abortPipeline: true
```

If the Quality Gate fails, Jenkins stops the pipeline.

15. What happens if Coverage is only 40%?

If the Quality Gate requires:

```
Coverage > 80%
```

Then:

```
Quality Gate Failed
```

```
Deployment Stops
```

16. What if the SonarQube Server is Down?

The analysis cannot be uploaded.

The pipeline typically fails unless configured to continue without SonarQube.

17. Why do we use `waitForQualityGate()` ?

Because SonarQube analysis is asynchronous.

Flow:

- Scanner uploads the analysis.
 - SonarQube processes it.
 - Jenkins waits for the result.
 - Pipeline continues only if the Quality Gate passes.
-

18. What is Sonar Scanner?

Sonar Scanner is the client that sends analysis data to the SonarQube server.

Example:

```
mvn sonar:sonar
```

19. Can SonarQube analyze multiple languages?

Yes.

It supports languages such as:

- Java
 - JavaScript
 - TypeScript
 - Python
 - Kotlin
 - Go
 - PHP
 - C#
 - C/C++ (commercial editions)
-

20. What metrics do you monitor?

Typically:

- Bugs
 - Vulnerabilities
 - Code Smells
 - Code Coverage
 - Code Duplication
 - Technical Debt
 - Reliability Rating
 - Maintainability Rating
 - Security Rating
-

21. Where is analysis history stored?

Analysis history is stored in the SonarQube database (typically PostgreSQL in production).

22. What is New Code vs Overall Code?

Large enterprise projects often contain legacy issues.

Instead of fixing everything, Quality Gates generally focus on **New Code**, ensuring that newly developed code meets quality standards while legacy issues are addressed gradually.

23. How is SonarQube integrated into CI/CD?

Typical flow:

```
Git Push
  ↓
Jenkins
  ↓
Compile
  ↓
Unit Tests
  ↓
JaCoCo
  ↓
SonarQube Scan
  ↓
Quality Gate
  ↓
Package
  ↓
Deploy
```

24. Can developers ignore issues?

Yes.

Issues can be marked as:

- False Positive

- Won't Fix
- Accepted (with proper justification)

Most organizations have governance around these decisions.

25. How do you exclude files from analysis?

Example:

```
sonar.exclusions=**/generated/**,**/*DTO.java
```

26. How do you analyze only one branch?

Example:

```
mvn sonar:sonar -Dsonar.branch.name=feature/payment
```

(Branch analysis depends on the SonarQube edition.)

27. How does Pull Request analysis work?

When a Pull Request is created:

- Jenkins triggers the build.
 - SonarQube analyzes only the changed code.
 - New bugs, vulnerabilities, and coverage are reported.
 - The Pull Request can be blocked if the Quality Gate fails.
-

28. How do you improve SonarQube scores?

- Fix bugs and vulnerabilities.
 - Refactor code smells.
 - Increase unit test coverage.
 - Reduce duplicated code.
 - Simplify complex methods.
 - Remove unused code.
-

Real Enterprise Interview Questions

- Explain your team's SonarQube setup.
 - What happens after `mvn sonar:sonar`?
 - How does Jenkins know the Quality Gate result?
 - Why do we use JaCoCo with SonarQube?
 - How do you troubleshoot a failed Quality Gate?
 - What metrics would stop a production deployment?
 - How do you handle legacy code with thousands of existing issues?
 - Have you customized a Quality Profile or Quality Gate?
 - How do you analyze Pull Requests in SonarQube?
 - What is the difference between SonarQube Community, Developer, and Enterprise editions?
-

Interview Summary

For experienced Java backend interviews (7–10 years), interviewers expect you to explain the complete workflow:

Git Commit → Jenkins → Maven → JUnit → JaCoCo → SonarQube → Quality Gate → Docker Build → Deployment

You should clearly understand the role of each component, how they integrate, and how the Quality Gate protects production deployments.